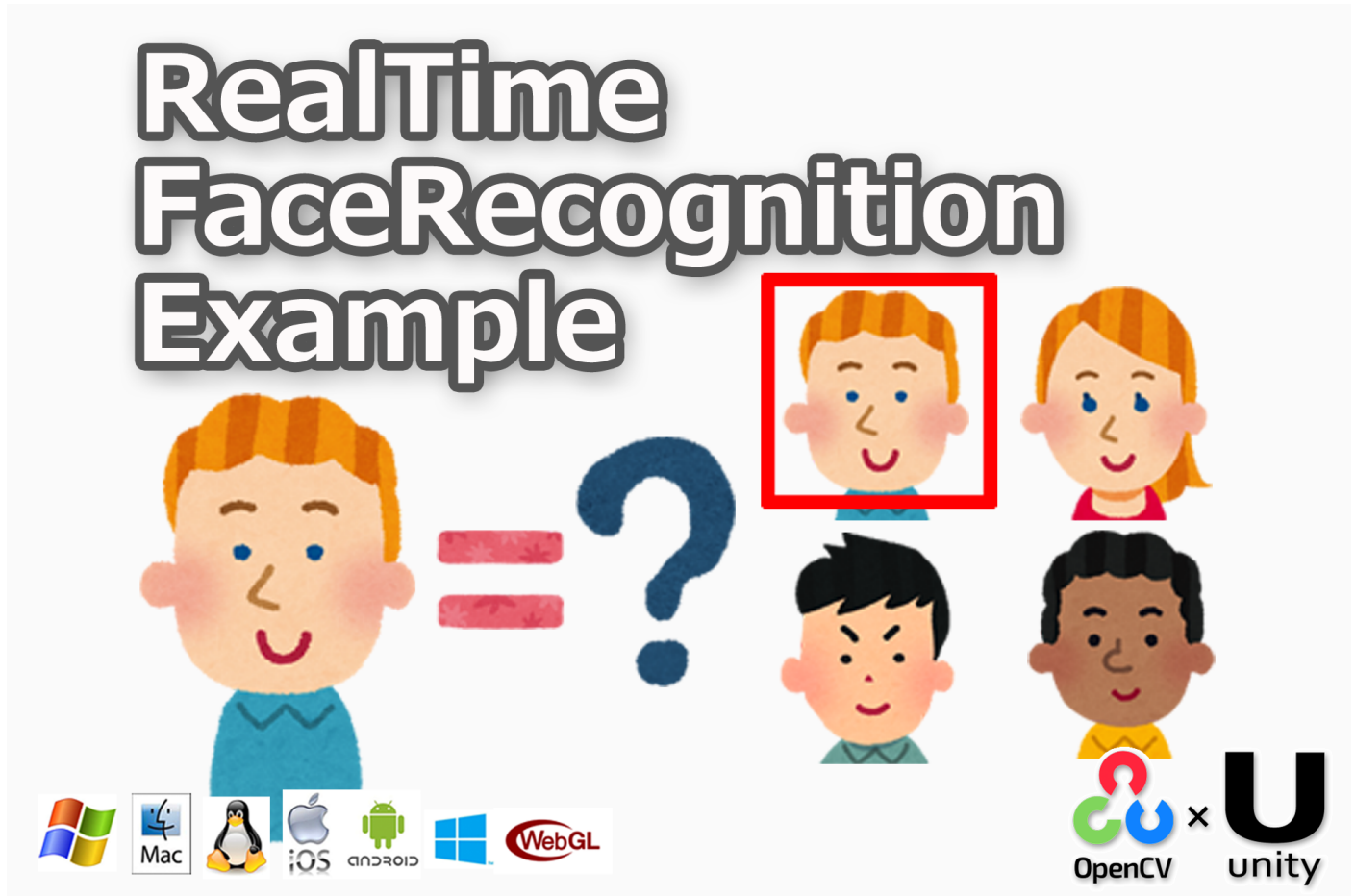


RealTime FaceRecognition

Example 1.1.3



About RealTime FaceRecognition Example

RealTime FaceRecognition Example is an **example project** demonstrating real-time face recognition using [OpenCV for Unity](#). This project showcases how to implement a complete face recognition system with **4-step procedure**.

The execution of this asset is required [OpenCV for Unity](#).



Our Asset Features



Cross Platform Support

- **Mobile:** iOS & Android
- **Desktop:** Windows & macOS & Linux Standalone support
- **Web:** WebGL support
- **ChromeOS:** ChromeOS support
- **XR:** visionOS support (beta) & Windows10 UWP support
- **Editor:** Support for preview in the Editor

Face Recognition Process

The face recognition procedure consists of **4 steps**:

1. **Face Detection** - Detect faces in the input image/video stream
2. **Face Preprocessing** - Prepare face images for recognition
3. **Collect and Learn Faces** - Train the recognition system with known faces
4. **Face Recognition** - Identify and match detected faces

Technical Implementation

- **Code Rewrite:** This project's code is a rewrite of [MasteringOpenCV Chapter8_FaceRecognition](#) using [OpenCV for Unity](#)
- **Real-time Processing:** Optimized for real-time face recognition performance
- **Easy Integration:** Designed to work seamlessly with Unity's development environment

Quick Links

- **Official Site** - Main product page
- **Example Code** - GitHub repository with examples
- **Android Demo** - Downloadable Android demo
- **WebGL Demo** - Online WebGL demo
- **Demo Video** - YouTube demonstration video

System Requirements

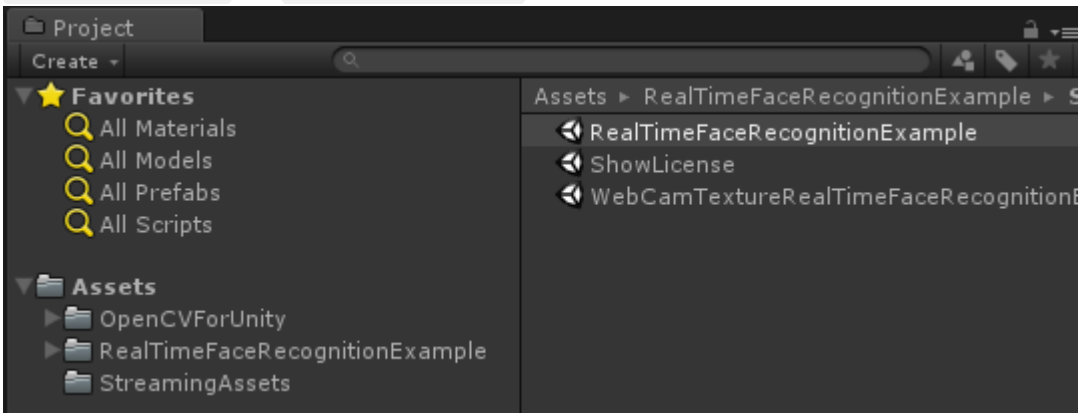
Platform	Requirement
Windows Standalone & Editor	Windows 8 or later

Platform	Requirement
Mac Standalone & Editor	macOS 10.13 or later
Linux Standalone & Editor	Ubuntu 18.04 or later
Android	API level 24 or later
iOS	iOS Version 12.0 or later
visionOS (beta)	visionOS 1 or later

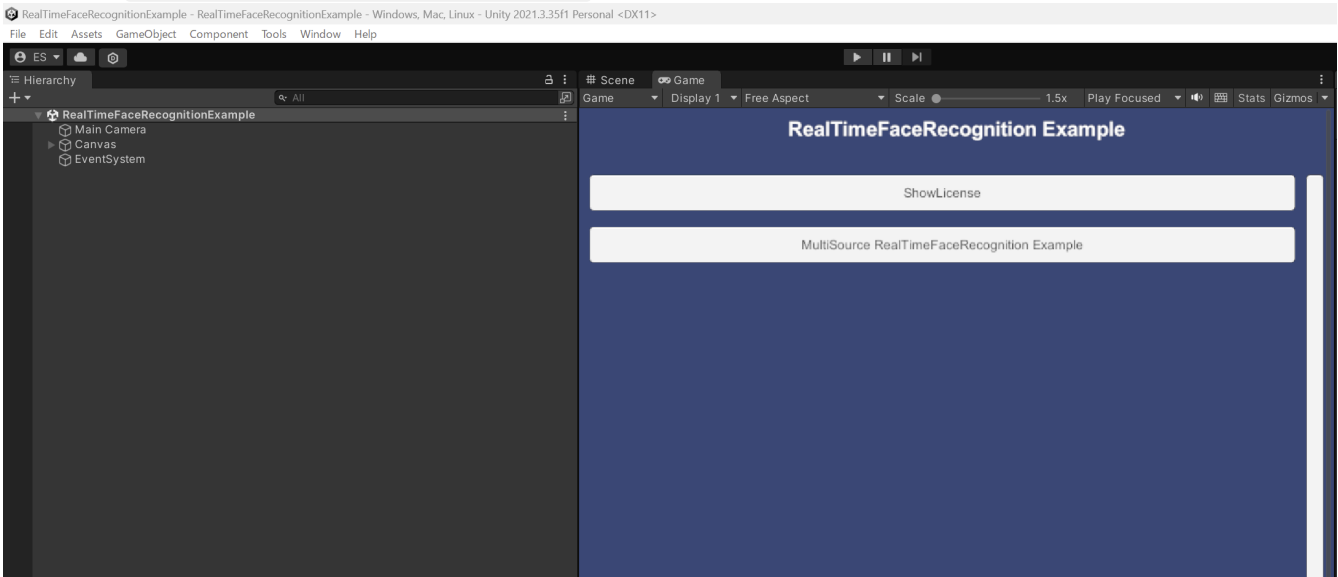
Setup Guide for Getting Started

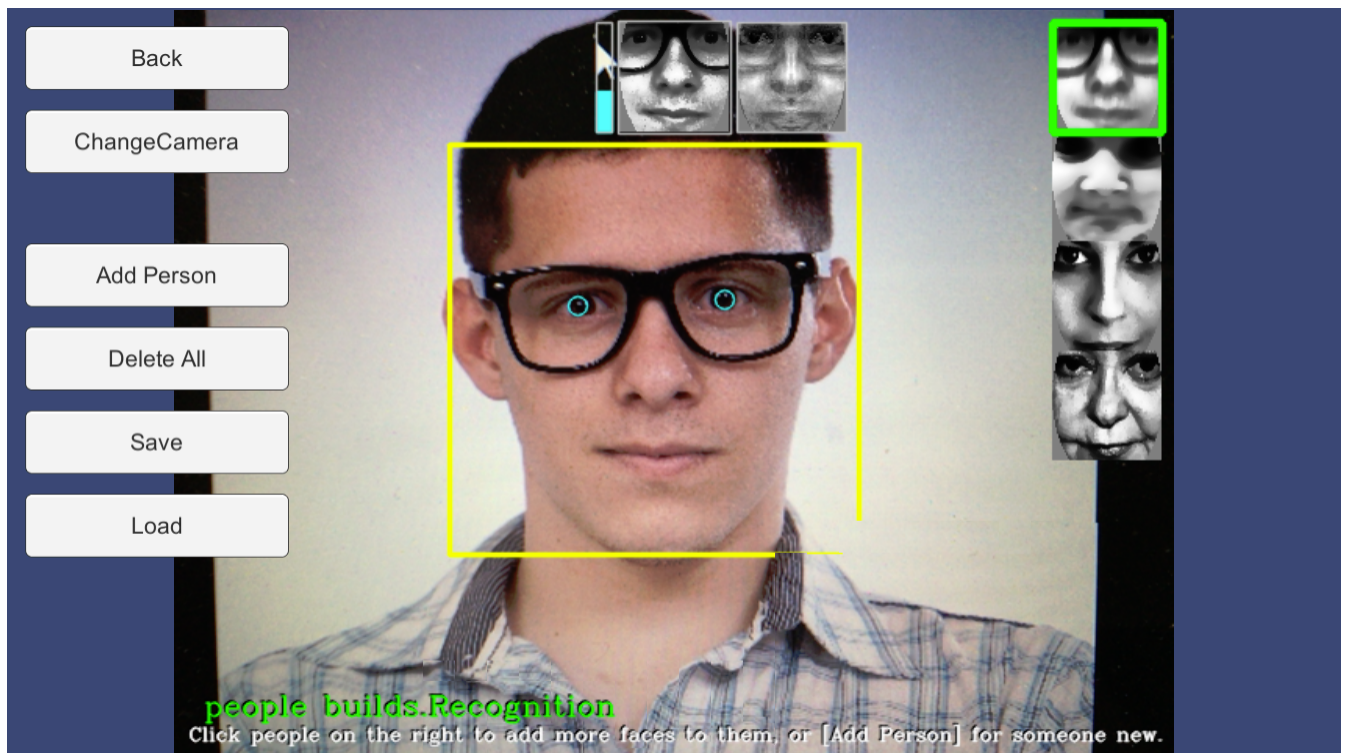
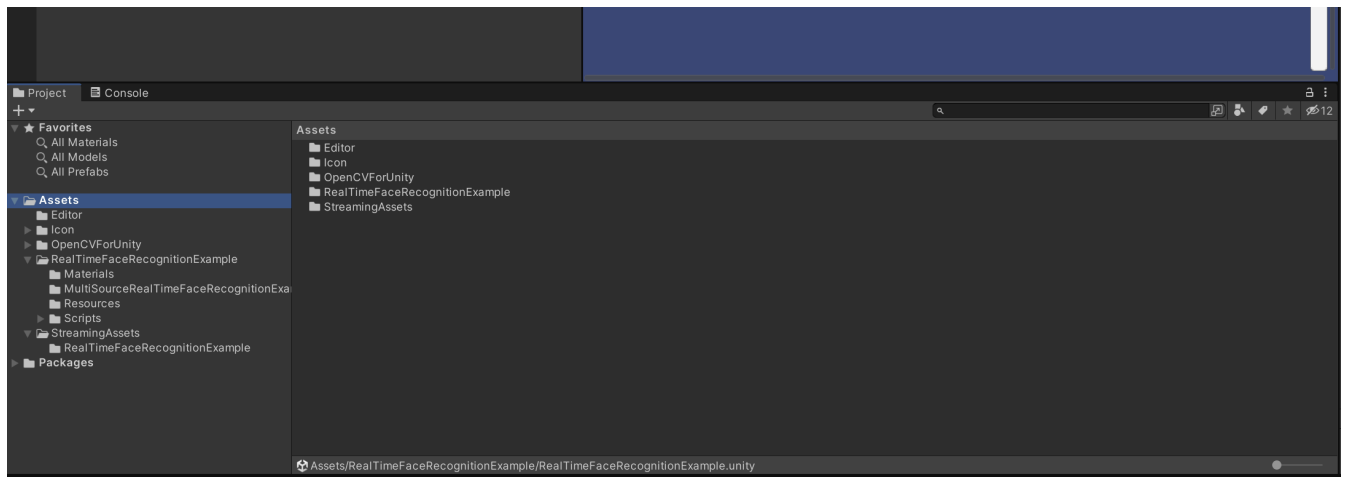
Quick Setup Procedure

1. Import [OpenCV for Unity](#) from the Package Manager .
2. Import [RealTime FaceRecognition Example](#) from the Package Manager .
3. Add all of the `***.unity` files in the `RealTimeFaceRecognitionExample/Scenes` folder to `Build Settings` → `Scenes In Build` .

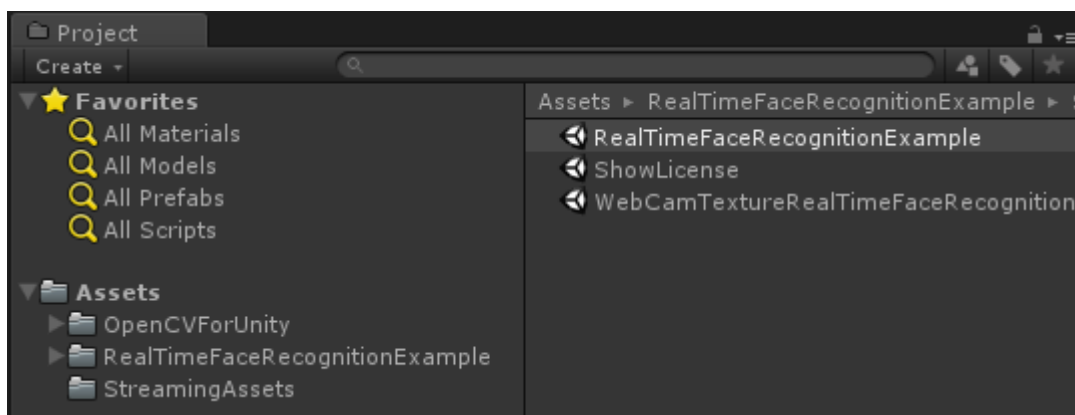


4. Run the `RealTimeFaceRecognitionExample` scene.





Screenshot after the Setup



Version Changes

1.1.3 [Common] Updated for OpenCV for Unity v3.0.4. (This asset requires OpenCVforUnity 3.0.4 or later.)

1.1.2 [Common] Updated for OpenCV for Unity v3.0.0. (This asset requires OpenCVforUnity 3.0.0 or later.)

1.1.1 [Common] Updated for OpenCV for Unity v2.6.4. (This asset requires OpenCVforUnity 2.6.4 or later.)

1.1.0 [Common] Changed the minimum supported version to Unity2021.3.35f1. [Common] Separated the examples using the Built-in Render Pipeline and Scriptable Render Pipeline.

1.0.9 [Common] Updated for OpenCV for Unity v2.5.9. (This asset requires OpenCVforUnity 2.5.9 or later.)

1.0.8 [Common] Updated for OpenCV for Unity v2.5.0. (This asset requires OpenCVforUnity 2.5.0 or later.)

1.0.7 [Common] Updated for OpenCV for Unity v2.4.2. (This asset requires OpenCVforUnity 2.4.2 or later.) [Common] Refactored the script.

1.0.6 [Common] Updated for OpenCV for Unity v2.3.8. (This asset requires OpenCVforUnity 2.3.8 or later.)

1.0.5 [Common] Updated for OpenCV for Unity v2.3.3. (This asset requires OpenCVforUnity 2.3.3 or later.)

1.0.4 [Common] Fixed save and load process. [Common] Update to WebCamTextureToMatHelper v1.0.6.

1.0.3 [Common] Updated for OpenCV for Unity v2.2.1. (This asset requires OpenCVforUnity 2.2.1 or later.)

1.0.2 [UWP] Fixed for UWP.

1.0.1 [Common] Changed the name of asset project. ("Sample" to "Example") [Common] Fixed WebCamTextureHelper.cs.

1.0.0 Initial version